24

Design Pattern

cheatsheet

BEHNAM MOHAMMADI

behnamm92@gmail.com

@bmohammadi

Abstract Factory

The Abstract Factory design pattern provides an interface for creating families of related or dependent objects without specifying their concrete classes.

@bmohammadi

Builder

The Builder design pattern separates the construction of a complex object from its representation so that the same construction process can create different representations.

@bmohammadi

Factory Method

The Factory Method design pattern defines an interface for creating an object, but let subclasses decide which class to instantiate. This pattern lets a class defer instantiation to subclasses.

@bmohammadi

Prototype

The Prototype design pattern specifies the kind of objects to create using a prototypical instance, and create new objects by copying this prototype.

@bmohammadi

Singleton

The Singleton design pattern ensures a class has only one instance and provide a global point of access to it.

@bmohammadi

Adapter

The Adapter design pattern converts the interface of a class into another interface clients expect. This design pattern lets classes work together that couldn‘t otherwise because of incompatible interfaces.

@bmohammadi

Bridge

The Bridge design pattern decouples an abstraction from its implementation so that the two can vary independently.

@bmohammadi

Composite

The Composite design pattern composes objects into tree structures to represent part-whole hierarchies. This pattern lets clients treat individual objects and compositions of objects uniformly.

@bmohammadi

Decorator

The Decorator design pattern attaches additional responsibilities to an object dynamically. This pattern provide a flexible alternative to subclassing for extending functionality.

@bmohammadi

Facade

The Facade design pattern provides a unified interface to a set of interfaces in a subsystem. This pattern defines a higher-level interface that makes the subsystem easier to use.

@bmohammadi

Flyweight

The Flyweight design pattern uses sharing to support large numbers of fine-grained objects efficiently.

@bmohammadi

Proxy

The Proxy design pattern provides a surrogate or placeholder for another object to control access to it.

@bmohammadi

Chain of Responsibility

The Chain of Responsibility design pattern avoids coupling the sender of a request to its receiver by giving more than one object a chance to handle the request. This pattern chains the receiving objects and passes the request along the chain until an object handles it.

@bmohammadi

Command

The Command design pattern encapsulates a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations.

@bmohammadi

Interpreter

Given a language, the Interpreter design pattern defines a representation for its grammar along with an interpreter that uses the representation to interpret sentences in the language.

@bmohammadi

Iterator

The Iterator design pattern provides a way to access the elements of an aggregate object sequentially without exposing its underlying representation.

@bmohammadi

Mediator

The Mediator design pattern defines an object that encapsulates how a set of objects interact. Mediator promotes loose coupling by keeping objects from referring to each other explicitly, and it lets you vary their interaction independently.

@bmohammadi

Memento

The Memento design pattern without violating encapsulation, captures and externalizes an object‘s internal state so that the object can be restored to this state later.

@bmohammadi

Observer

The Observer design pattern defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

@bmohammadi

State

The State design pattern allows an object to alter its behavior when its internal state changes. The object will appear to change its class.

@bmohammadi

State

The State design pattern allows an object to alter its behavior when its internal state changes. The object will appear to change its class.

@bmohammadi

Strategy

The Strategy design pattern defines a family of algorithms, encapsulate each one, and make them interchangeable. This pattern lets the algorithm vary independently from clients that use it.

@bmohammadi

Template Method

The Template Method design pattern defines the skeleton of an algorithm in an operation, deferring some steps to subclasses. This pattern lets subclasses redefine certain steps of an algorithm without changing the algorithm‘s structure.

@bmohammadi

Visitor

The Visitor design pattern represents an operation to be performed on the elements of an object structure. This pattern lets you define a new operation without changing the classes of the elements on which it operates.

@bmohammadi